

Regular Expressions and DFAs

Regular Expression

- Notation to specify a language
 - Declarative
 - Sort of like a programming language.
 - Fundamental in some languages like perl and applications like grep or lex
 - Capable of describing the same thing as a NFA
 - The two are actually equivalent, so $RE = NFA = DFA$
 - We can define an algebra for regular expressions
-

Algebra for language

- Previously we discussed these operators:
 - Union
 - Concatenation
 - Kleene Star

Remember

* has a higher precedence than concatenation and
concatenation has a higher precedence than +

RE Examples

$$L(\mathbf{001}) = \{001\}$$

$$L(\mathbf{0+10^*}) = \{0, 1, 10, 100, 1000, 10000, \dots\}$$

$$L(\mathbf{0^*10^*}) = \{1, 01, 10, 010, 0010, \dots\} \quad \text{i.e. } \{w \mid w \text{ has exactly a single } 1\}$$

$$L(\mathbf{\Sigma\Sigma}^*) = \{w \mid w \text{ is a string of even length}\}$$

$$L(\mathbf{(0(0+1))^*}) = \{\epsilon, 00, 01, 0000, 0001, 0100, 0101, \dots\}$$

$$L(\mathbf{(0+\epsilon)(1+\epsilon)}) = \{\epsilon, 0, 1, 01\}$$

$$L(1\emptyset) = \emptyset \quad ; \text{ concatenating the empty set to any set yields the empty set.}$$

$$R\epsilon = R$$

$$R+\emptyset = R$$

Exercise 1

- Let Σ be a finite set of symbols
- $\Sigma = \{10, 11\}$, $\Sigma^* = ?$

Answer:

$$\Sigma^* = \{\epsilon, 10, 11, 1010, 1011, 1110, 1111, \dots\}$$

Exercises 2

- $L1 = \{10, 1\}$, $L2 = \{011, 11\}$, $L1L2 = ?$
- $L1L2 = \{10011, 1011, 111\}$

Exercises 3

■ Write RE for

- All strings of 0's and 1's

- $(0|1)^*$

- All strings of 0's and 1's with at least 2 consecutive 0's

- $(0|1)^*00(0|1)^*$

- All strings of 0's and 1's beginning with 1 and not having two consecutive 0's

- $(1+10)^*$

More Exercises

- All strings of 0's and 1's ending in 011
 $(0|1)^*011$
 - any number of 0's followed by any number of 1's followed by any number of 2's
 $0^*1^*2^*$
 - strings of 0,1,2 with at least one of each symbol
 $00^*11^*22^*$
-

More Exercise

- The set of all strings whose number of 0's is divisible by 3
 - $(1+01^*01^*0)^*$

Exercise

1. It can generate only one a or it can generate only one b
2. language of all those strings starting with a, defined over $\Sigma = \{a, b\}$

1. $(a + b)$

2. $a(a+b)^*$

Exercise

1. Set of strings of a's and b's of any length including the null string. So $L = \{ \epsilon, a, b, aa, ab, bb, ba, aaa, \dots \}$
2. Set of strings of a's and b's ending with the string **abb**. So $L = \{ abb, aabb, babb, aaabb, ababb, \dots \}$
3. Set consisting of even number of 1's including empty string, So $L = \{ \epsilon, 11, 1111, 111111, \dots \}$, defined over $\Sigma = \{1\}$

1. $(a+b)^*$
2. $(a+b)^*abb$
3. $(11)^*$

Exercise

1. Set consisting of even number of 1's including empty string, defined over $\Sigma = \{0,1\}$
2. Set of strings consisting of even number of a's followed by odd number of b's, so $L = \{b, aab, aabbb, aabbbbb, aaaab, aaaabbb, \dots\}$
3. String of a's and b's of even length can be obtained by concatenating any combination of the strings aa, ab, ba and bb including null, so $L = \{aa, ab, ba, bb, aaab, aaba, \dots\}$

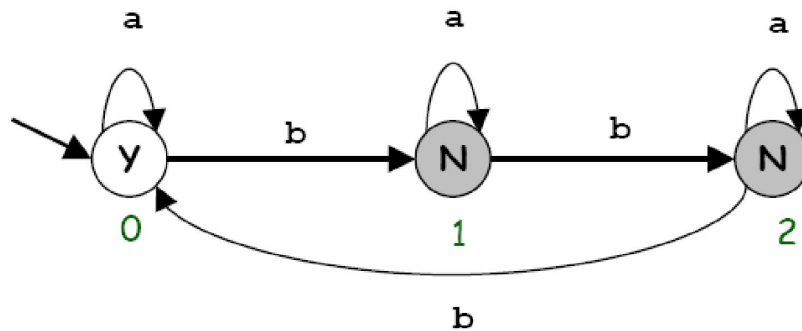
1. $(0^*10^*10^*)^*$
2. $(aa)^*(bb)^*b$
3. $(aa + ab + ba + bb)^*$

Theory of DFAs and REs

- RE. Concise way to describe a set of strings.
- DFA. Machine to recognize whether a given string is in a given set.
- **Duality**: for any DFA, there exists a regular expression to describe the same set of strings; for any regular expression, there exists a DFA that recognizes the same set.

Duality Example

- DFA for multiple of 3 b's:

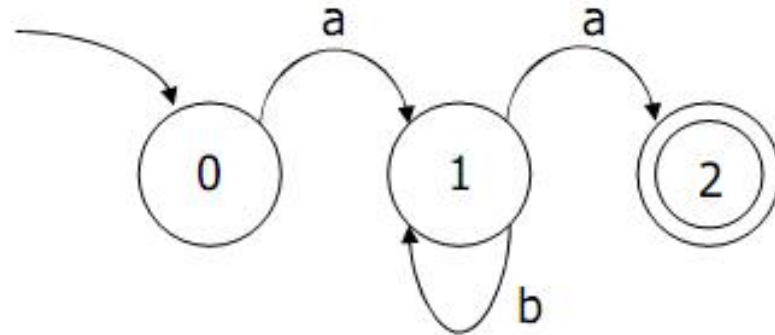


- RE for multiple of 3 b's:

$$(a^*ba^*ba^*ba^*)^* a^*$$

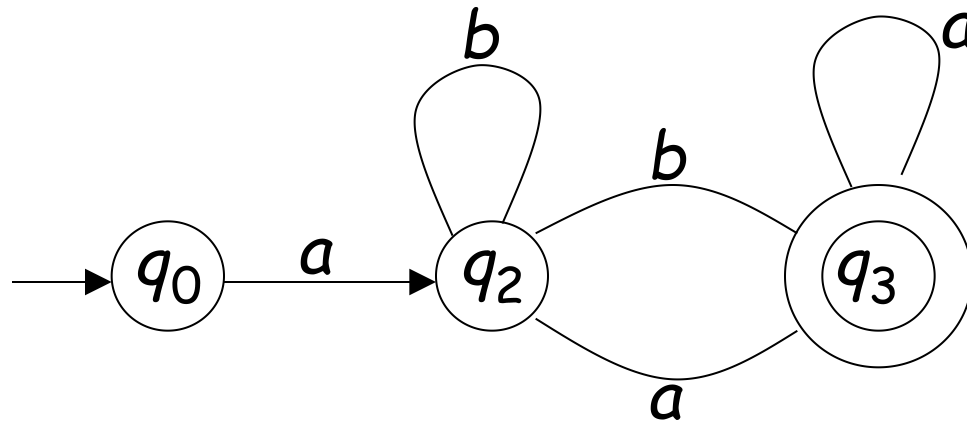
Problem 1

- Make a DFA that accepts the strings in the language denoted by regular expression ab^*a



Problem 2

- Write the RE for the following automata:



- $a(a|b)^*a$

DFA to RE: State Elimination

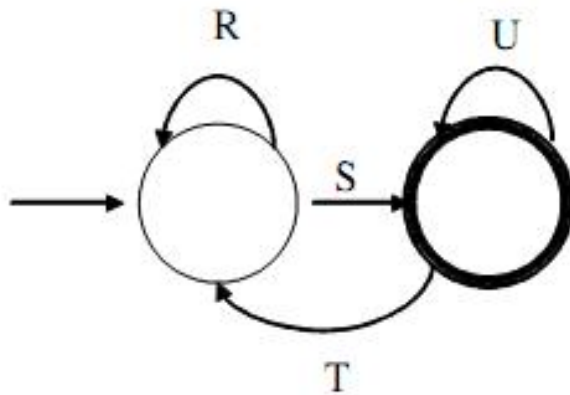
- Eliminates states of the automaton and replaces the edges with regular expressions that includes the behavior of the eliminated states.
- Eventually we get down to the situation with just a start and final node, and this is easy to express as a RE

DFA to RE via State Elimination (1)

- Starting with intermediate states and then moving to accepting states, apply the state elimination process to produce an equivalent automaton with regular expression labels on the edges.
- The result will be one or two state automaton with a start state and accepting state.

DFA to RE State Elimination (2)

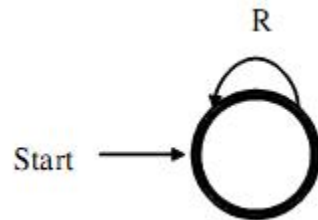
- If the two states are different, we will have an automaton that looks like the following:



- We can describe this automaton as: $(R \mid SU^*T)^*SU^*$

DFA to RE State Elimination (3)

- If the start state is also an accepting state, then we must also perform a state elimination from the original automaton that gets rid of every state but the start state. This leaves the following:



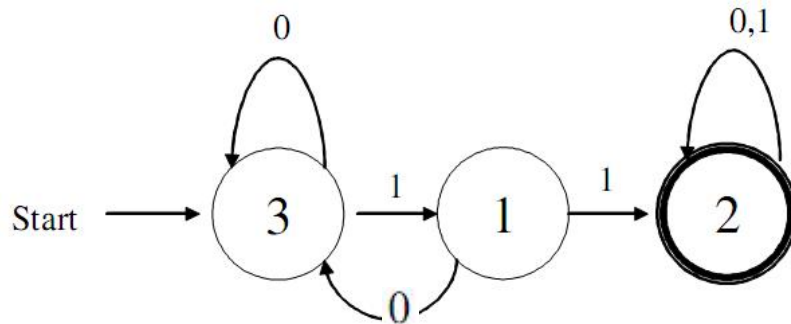
- We can describe this automaton as simply R^*

DFA to RE State Elimination (4)

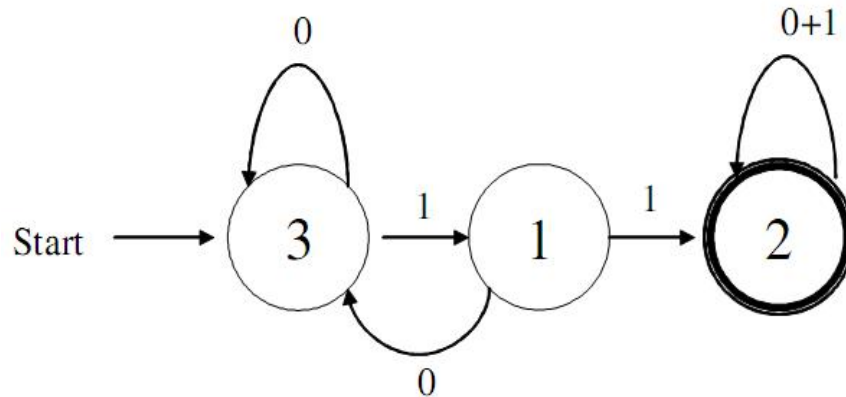
- If there are n accepting states, we must repeat the above steps for each accepting states to get n different regular expressions, $R_1, R_2, \dots R_n$.
- For each repeat we turn any other accepting state to non-accepting.
- The desired regular expression for the automaton is then the union of each of the n regular expressions: $R_1 \cup R_2 \dots \cup R_N$

DFA->RE Example

- Convert the following to a RE:

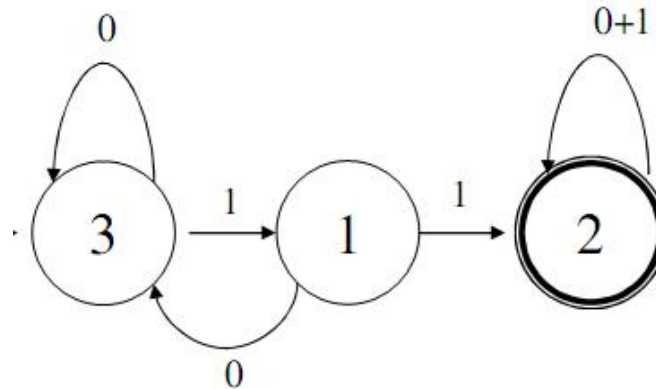


- First convert the edges to RE's:

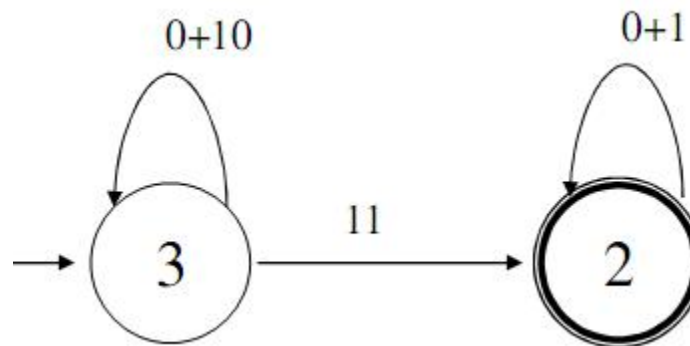


DFA $\rightarrow$ RE Example (2)

- Eliminate State 1:



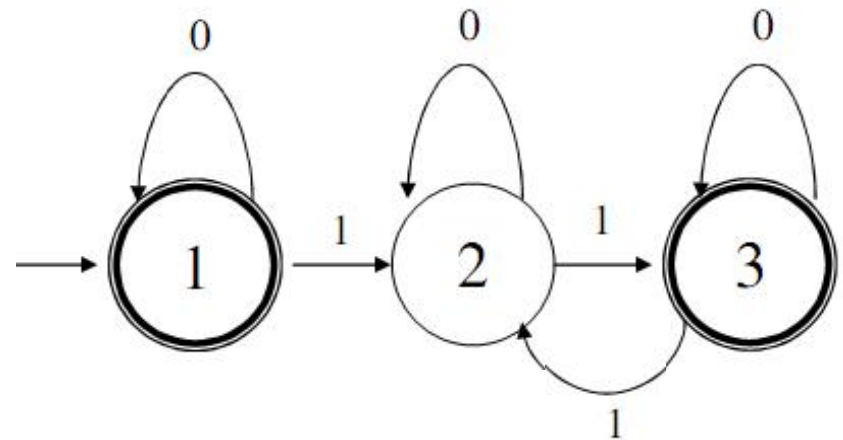
- Note edge from 3 $\rightarrow$ 3



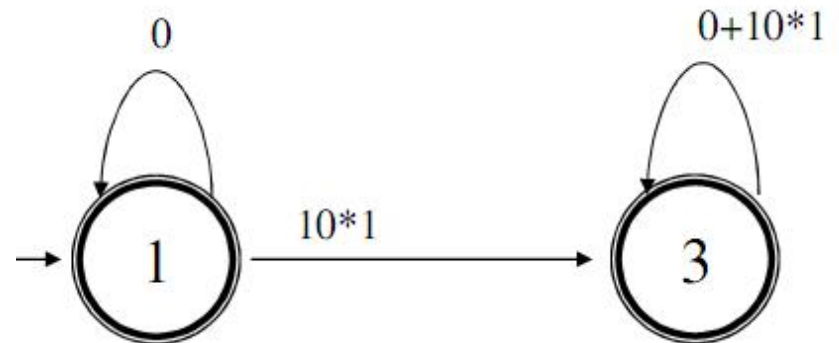
- Answer: $(0+10)^*11(0+1)^*$

Second Example

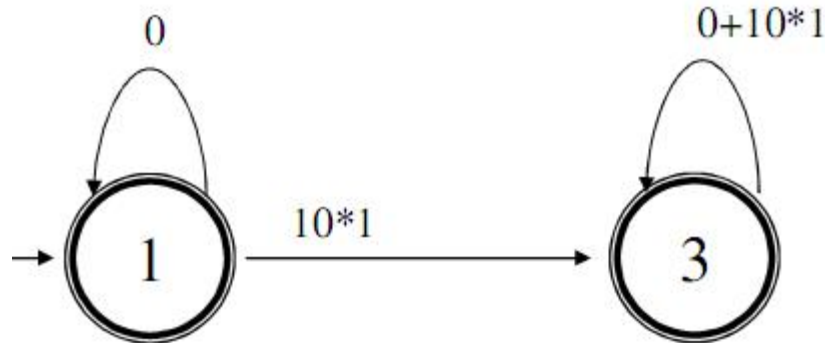
- Automata that accepts even number of 1's



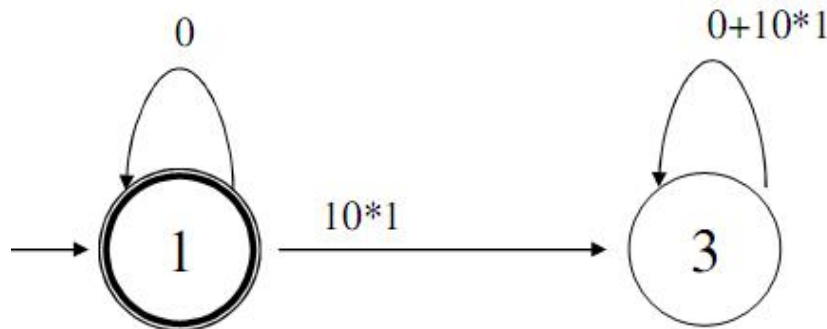
- Eliminate state 2:



Second Example (2)

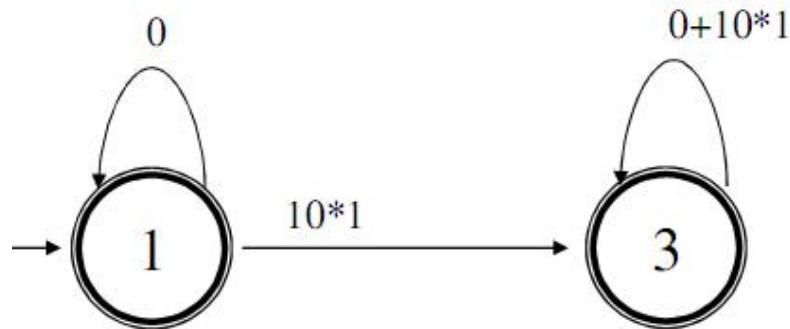


- Two accepting states, turn off state 3 first

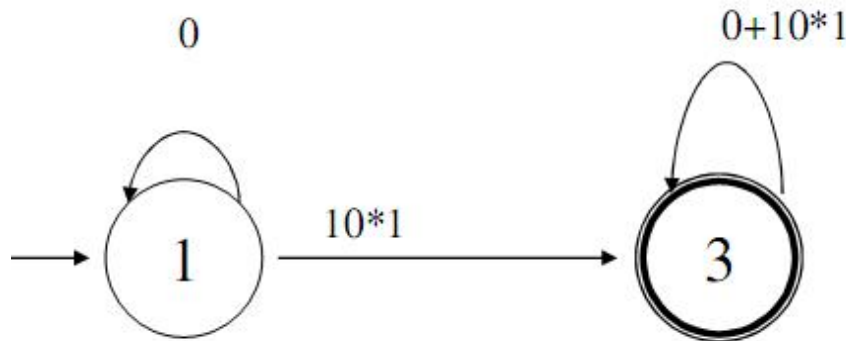


- This is just 0^* ; can ignore going to state 3 since we would “die”

Second Example (3)



- Turn off state 1 second:



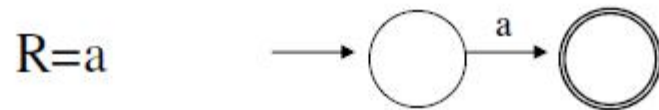
- This is just $0^*10^*1(0|10^*1)^*$
- Combine from previous slide to get $0^* | 0^*10^*1(0|10^*1)^*$

RE $\rightarrow$ Automata

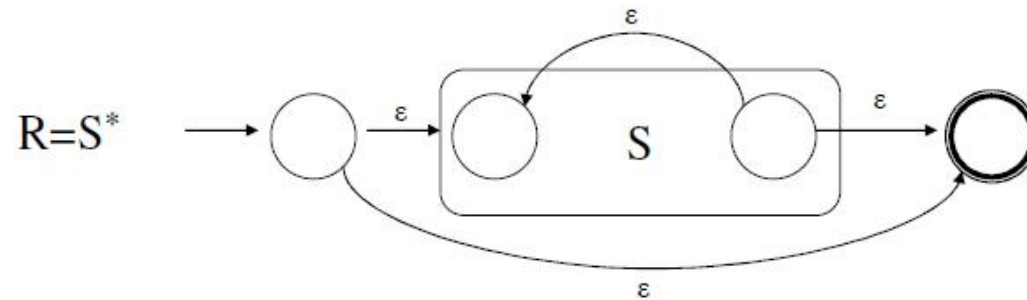
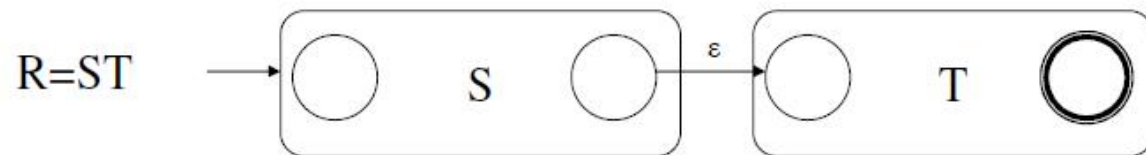
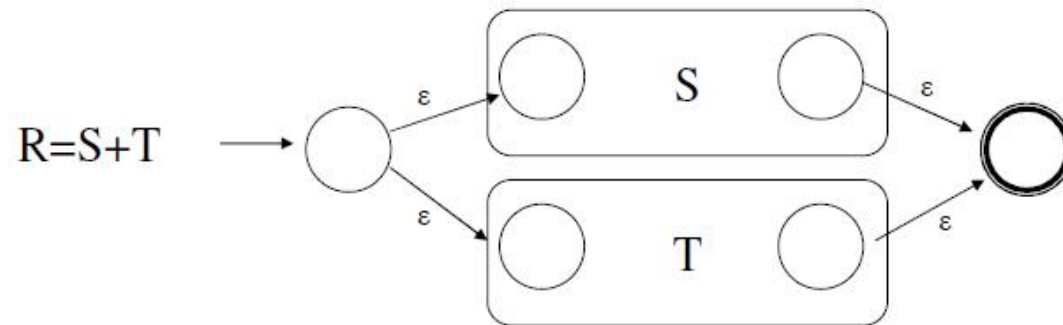
- We have shown we can convert an automata to a RE. To show equivalence we must also go the other direction, convert a RE to an automaton.
- We can do this easiest by converting a RE to an ϵ -NFA
 - Inductive construction
 - Start with a simple basis, use that to build more complex parts of the NFA

RE $\rightarrow$ Automata

- Basis:

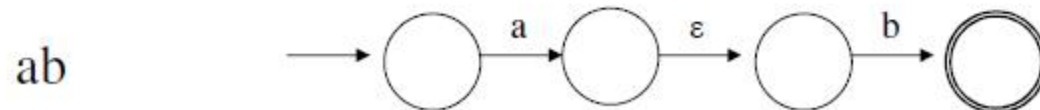
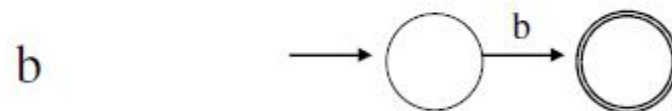
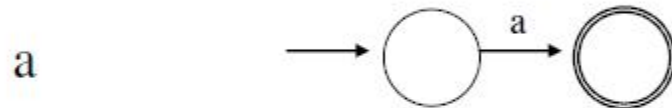


RE $\rightarrow$ Automata



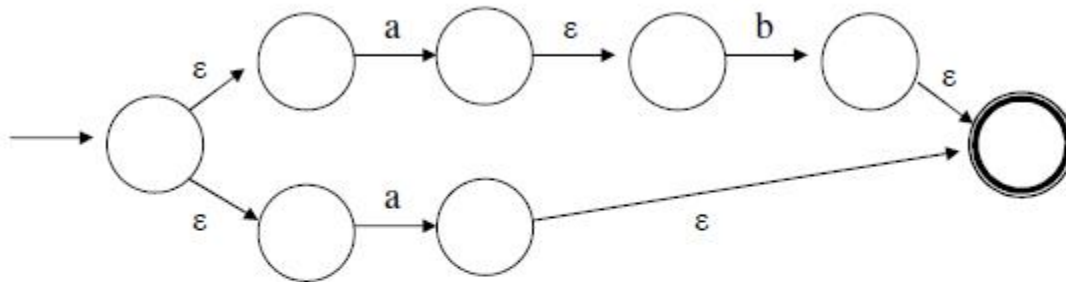
RE $\rightarrow$ Automata

- Convert $R = (ab+ a)^*$ to an NFA
 - We proceed in stages, starting from simple elements and working our way up

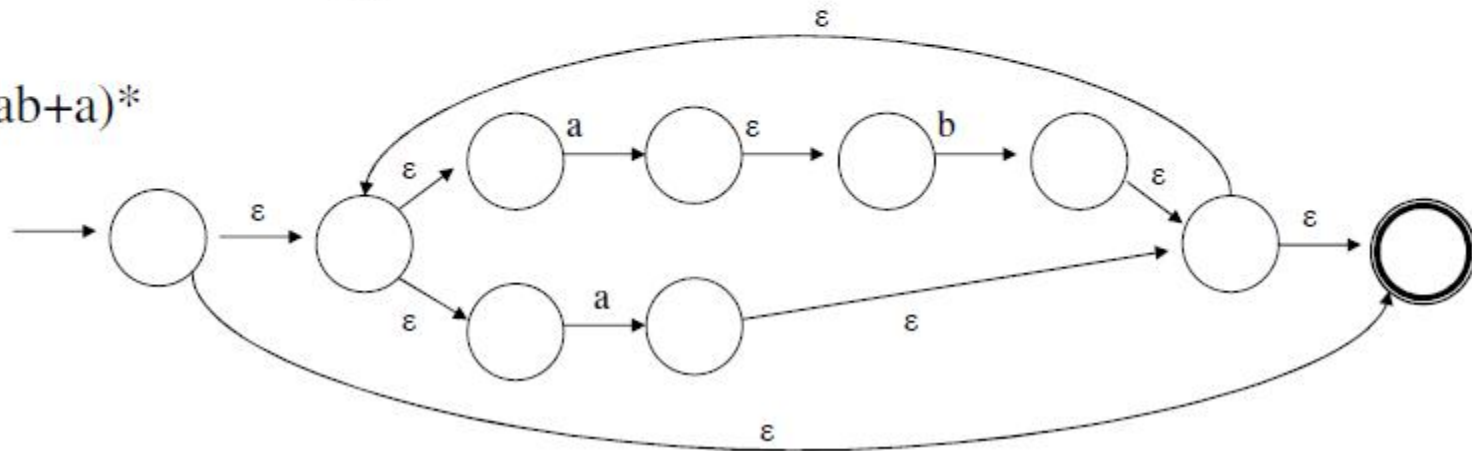


RE $\rightarrow$ Automata

$ab+a$



$(ab+a)^*$



RE $\rightarrow$ Automata

- Another approach
 - Mishra